



Universidad Tecnológica Nacional

Tecnicatura universitaria en programación

Programación II

Trabajo practico integrador

"Sistema Food Store"

Alumno: Tomás Lucas

Fecha de entrega: 22/06/2026

Contenido

1. Marco teórico.....	3
2. Decisiones técnicas y arquitectura.....	3
2.1 Enfoque de la solución	3
2.2 Organización de paquetes.....	3
2.3 Validación inicial mediante TestApp	5
2.3.1 Entidades Basica.....	5
2.3.2 CRUD de categorías.....	5
2.3.3 CRUD de productos	7
2.3.4 CRUD de Usuarios.....	8
2.3.5 Prueba integral del sistema.....	9
2.3.6 Cambio de estado del pedido	12
2.3.7 Creación y navegación por el menú	13
2.3.8 Navegación por menú	14
2.3.9 Navegación final por menú	15
2.3.10 Creación de usuario.....	15
2.3.11 Creación de usuario.....	16
2.3.12 Eliminación de usuario	16
2.3.13 Creación de pedido	17
2.3.14 Listado de pedidos	17
2.3.15 Estado de pedido.....	18
2.3.16 Eliminación de pedido	18
3. Dificultades y soluciones.....	19
4. Bibliografía / Webgrafía.	20
5. Repositorio y video explicativo.	20

1. Marco teórico.

Para el desarrollo de este proyecto se utilizó el lenguaje de programación Java, aplicando los conceptos fundamentales de Programación Orientada a Objetos vistos durante la cursada.

Java es un lenguaje de programación orientado a objetos ampliamente utilizado para el desarrollo de aplicaciones debido a su portabilidad, robustez y facilidad para organizar el código mediante clases y objetos. En este trabajo se utilizó Apache NetBeans como entorno de desarrollo integrado (IDE) para la creación y ejecución del sistema.

Uno de los conceptos principales aplicados fue la Programación Orientada a Objetos (POO), que permite modelar entidades del mundo real mediante clases. A través de este paradigma se trabajó con conceptos como encapsulamiento, herencia, abstracción y reutilización de código.

También se utilizaron colecciones dinámicas mediante la clase ArrayList para almacenar la información del sistema durante la ejecución del programa. Estas estructuras permitieron gestionar categorías, productos, usuarios y pedidos de manera flexible.

Para representar conjuntos de valores predefinidos se implementaron enumerados (enum), utilizados para los roles de usuario, estados de los pedidos y formas de pago.

Además, se desarrollaron excepciones personalizadas con el objetivo de controlar errores específicos del sistema, como la búsqueda de entidades inexistentes, correos electrónicos duplicados o valores de stock inválidos.

Por otra parte, se implementó una interfaz denominada Calculable, utilizada por la entidad Pedido para realizar el cálculo automático del total de cada compra. Esto permitió aplicar conceptos de abstracción y reutilización de funcionalidades.

Cabe destacar que el sistema no utiliza una base de datos relacional, ya que la información se almacena temporalmente en memoria mediante colecciones ArrayList. Sin embargo, la estructura desarrollada permite una futura integración con motores de bases de datos como MySQL o PostgreSQL.

2. Decisiones técnicas y arquitectura.

2.1 Enfoque de la solución

Para el desarrollo del proyecto se utilizó el paradigma de Programación Orientada a Objetos en Java. La solución fue diseñada siguiendo una estructura por capas, separando las entidades del negocio, los servicios y la interfaz de usuario.

El sistema fue desarrollado de manera incremental. Inicialmente se realizaron pruebas unitarias utilizando la clase TestApp para validar el funcionamiento de las entidades y servicios de forma aislada. Una vez verificadas las funcionalidades principales, se implementó una interfaz por consola mediante menús interactivos que permiten gestionar categorías, productos, usuarios y pedidos.

2.2 Organización de paquetes

La aplicación fue organizada en distintos paquetes para mantener una estructura clara y facilitar el mantenimiento del código:

entities

Contiene las clases que representan los objetos principales del sistema:

- Base
- Categoria
- Producto
- Usuario
- Pedido
- DetallePedido

La clase Base fue utilizada como superclase para centralizar atributos comunes como id y eliminado.

services

Contiene la lógica de negocio del sistema:

- CategoriaService
- ProductoService
- UsuarioService
- PedidoService

Cada servicio administra las operaciones CRUD correspondientes a su entidad.

enums

Contiene los tipos enumerados utilizados por el sistema:

- Estado
- FormaPago
- Rol

Estos enumerados permiten restringir los valores posibles y mejorar la consistencia de los datos.

exceptions

Contiene las excepciones personalizadas:

- EntidadNoEncontradaException
- EmailDuplicadoException
- StockInvalidoException

Su implementación permitió validar situaciones de error específicas del sistema.

interfaces

Contiene la interfaz:

- Calculable

La misma fue implementada por la entidad Pedido para realizar el cálculo automático del total.

Ui (User Interface)

Contiene la interfaz de usuario por consola:

- Main
- TestApp

Desde este paquete se gestionan los menús y la interacción con el usuario.

2.3 Validación inicial mediante TestApp

2.3.1 Entidades Basica

Ilustración 1 y 2. Validación de las entidades Categoría, Producto y Usuario.

Se verificó el correcto funcionamiento de las entidades principales del sistema (Categoría, Producto y Usuario), comprobando la herencia desde la clase Base y la implementación de los métodos toString().

```
package FoodStore.ui;

import FoodStore.entities.Categoria;
import FoodStore.entities.Producto;
import FoodStore.entities.Usuario;
import FoodStore.enums.Rol;

public class TestApp {

    public static void main(String[] args) {

        Categoria bebidas =
            new Categoria(
                1L,
                "Bebidas",
                "Gaseosas y jugos");

        Producto coca =
            new Producto(
                1L,
                "Coca Cola",
                1500.0,
                20,
                bebidas);

        Usuario usuario =
            new Usuario(
                1L,
                "Tomas",
                "tomas@mail.com",
                "1234",
                Rol.USUARIO);

        System.out.println(bebidas);
        System.out.println(coca);
        System.out.println(usuario);
    }
}
```

Ilustración 1 Validación de las entidades Categoría, Producto y Usuario.

```
run:
Categoria{id=1, nombre='Bebidas', descripcion='Gaseosas y jugos'}
Producto{id=1, nombre='Coca Cola', precio=1500.0, stock=20, categoria=Bebidas}
Usuario{id=1, nombre='Tomas', email='tomas@mail.com', rol=USUARIO}
BUILD SUCCESSFUL (total time: 0 seconds)
```

Ilustración 2 salida por consola

2.3.2 CRUD de categorías

Ilustración 3, 4 y 5. Operaciones CRUD de Categorías.

Se realizó la prueba completa del CRUD de Categorías: alta, consulta, modificación y eliminación lógica. La baja lógica se implementó mediante el atributo eliminado heredado de la clase Base.

```
package FoodStore.ui;

import FoodStore.exceptions.EntidadNoEncontradaException;
import FoodStore.services.CategoriaService;

public class TestApp {

    public static void main(String[] args) {

        CategoriaService service =
            new CategoriaService();

        service.crearCategoria(
            "Bebidas",
            "Gaseosas y jugos");

        service.crearCategoria(
            "Comidas",
            "Comidas rápidas");

        System.out.println("LISTADO");

        service.listarCategorias();

        try {

            System.out.println("\nBUSQUEDA");

            System.out.println(
                service.buscarCategoriaPorId(
                    1L));

            System.out.println("\nACTUALIZACION");

            service.actualizarCategoria(
                1L,
                "Bebidas",
                "Bebidas frias");

            System.out.println(
                service.buscarCategoriaPorId(
                    1L));

        }

    }

}
```

Ilustración 3 Operaciones CRUD de categorías

```
        System.out.println("\nELIMINACION");

        service.eliminarCategoria(
            2L);

        service.listarCategorias();

    } catch (
        EntidadNoEncontradaException e) {

        System.out.println(
            e.getMessage());

    }

}
```

Ilustración 4 Operaciones CRUD de categorías

```

run:
LISTADO
Categoria{id=1, nombre='Bebidas', descripcion='Gaseosas y jugos'}
Categoria{id=2, nombre='Comidas', descripcion='Comidas rápidas'}

BUSQUEDA
Categoria{id=1, nombre='Bebidas', descripcion='Gaseosas y jugos'}

ACTUALIZACION
Categoria{id=1, nombre='Bebidas', descripcion='Bebidas frías'}

ELIMINACION
Categoria{id=1, nombre='Bebidas', descripcion='Bebidas frías'}
BUILD SUCCESSFUL (total time: 0 seconds)

```

Ilustración 5 salida por consola

2.3.3 CRUD de productos

Ilustración 6 y 7. Operaciones CRUD de Productos.

Se verificó el CRUD de Productos y la relación entre Producto y Categoría. Además, se implementó la excepción `StockInvalidoException` para validar que el stock no pueda ser negativo.

```

package FoodStore.ui;

import FoodStore.exceptions.EntidadNoEncontradaException;
import FoodStore.exceptions.StockInvalidoException;
import FoodStore.services.CategoriaService;
import FoodStore.services.ProductoService;

public class TestApp {

    public static void main(String[] args) {

        CategoriaService categoriaService =
            new CategoriaService();

        ProductoService productoService =
            new ProductoService();

        try {

            categoriaService.crearCategoria(
                "Bebidas",
                "Gaseosas y jugos");

            productoService.crearProducto(
                "Coca Cola",
                1500.0,
                20,
                categoriaService.buscarCategoriaPorId(
                    1L));

            productoService.crearProducto(
                "Sprite",
                1400.0,
                10,
                categoriaService.buscarCategoriaPorId(
                    1L));

            productoService.listarProductos();

        } catch (
            EntidadNoEncontradaException |
            StockInvalidoException e) {

            System.out.println(
                e.getMessage());

        }

    }

}

```

Ilustración 6 Operaciones CRUD de productos

```
run:
Producto{id=1, nombre='Coca Cola', precio=1500.0, stock=20, categoria=Bebidas}
Producto{id=2, nombre='Sprite', precio=1400.0, stock=10, categoria=Bebidas}
BUILD SUCCESSFUL (total time: 0 seconds)
```

Ilustración 7 salida por consola

2.3.4 CRUD de Usuarios

Ilustración 8 y 9. Operaciones CRUD de Usuarios.

Se probó el CRUD de Usuarios, incorporando el uso del enumerado Rol y la excepción EmailDuplicadoException para impedir registros con correos repetidos.

```
package FoodStore.ui;

import FoodStore.enums.Rol;
import FoodStore.exceptions.EmailDuplicadoException;
import FoodStore.services.UsuarioService;

public class TestApp {

    public static void main(String[] args) {

        UsuarioService usuarioService =
            new UsuarioService();

        try {

            usuarioService.crearUsuario(
                "Tomas",
                "tomas@gmail.com",
                "1234",
                Rol.ADMIN);

            usuarioService.crearUsuario(
                "Juan",
                "juan@gmail.com",
                "5678",
                Rol.USUARIO);

            usuarioService.listarUsuarios();

        } catch (
            EmailDuplicadoException e) {

            System.out.println(
                e.getMessage());

        }

    }

}
```

Ilustración 8 Operaciones CRUD de usuarios

```
run:
Usuario{id=1, nombre='Tomas', email='tomas@gmail.com', rol=ADMIN}
Usuario{id=2, nombre='Juan', email='juan@gmail.com', rol=USUARIO}
BUILD SUCCESSFUL (total time: 0 seconds)
```

Ilustración 9 salida por consola

Figura 10, 11, 12 y 13. Integración de Categorías, Productos, Usuarios y Pedidos.

Se realizó una prueba integral del sistema. Se crearon categorías, productos y usuarios, posteriormente se generó un pedido asociado a un usuario y se agregaron productos mediante `DetallePedido`. El total del pedido fue calculado utilizando la interfaz `Calculable` y se verificó el correcto funcionamiento de las relaciones entre las entidades.

```
package FoodStore.ui;

import FoodStore.entities.Usuario;
import FoodStore.enums.FormaPago;
import FoodStore.enums.Rol;
import FoodStore.exceptions.EmailDuplicadoException;
import FoodStore.exceptions.EntidadNoEncontradaException;
import FoodStore.exceptions.StockInvalidoException;
import FoodStore.services.CategoriaService;
import FoodStore.services.PedidoService;
import FoodStore.services.ProductoService;
import FoodStore.services.UsuarioService;

public class TestApp {

    public static void main(String[] args) {

        CategoriaService categoriaService =
            new CategoriaService();

        ProductoService productoService =
            new ProductoService();

        UsuarioService usuarioService =
            new UsuarioService();

        PedidoService pedidoService =
            new PedidoService();

        try {

            // =====
            // CATEGORIAS
            // =====

            categoriaService.crearCategoria(
                "Bebidas",
                "Gaseosas y jugos");

            categoriaService.crearCategoria(
                "Comidas",
                "Comidas rápidas");

            System.out.println(
                "\n===== CATEGORIAS =====");

            categoriaService.listarCategorias();
```

Ilustración 10 Prueba integral del sistema

```
// =====  
// PRODUCTOS  
// =====  
  
productoService.crearProducto(  
    "Coca Cola",  
    1500.0,  
    20,  
    categoriaService.buscarCategoriaPorId(1L));  
  
productoService.crearProducto(  
    "Sprite",  
    1400.0,  
    10,  
    categoriaService.buscarCategoriaPorId(1L));  
  
System.out.println(  
    "\n===== PRODUCTOS =====");  
  
productoService.listarProductos();  
  
// =====  
// USUARIOS  
// =====  
  
usuarioService.crearUsuario(  
    "Tomas",  
    "tomas@gmail.com",  
    "1234",  
    Rol.ADMIN);  
  
usuarioService.crearUsuario(  
    "Juan",  
    "juan@gmail.com",  
    "5678",  
    Rol.USUARIO);  
  
System.out.println(  
    "\n===== USUARIOS =====");  
  
usuarioService.listarUsuarios();
```

Ilustración 11 Prueba integral del sistema

```

// =====
// PEDIDOS
// =====

Usuario usuario =
    usuarioService.buscarUsuarioPorId(
        1L);

pedidoService.crearPedido(
    usuario,
    FormaPago.TARJETA);

pedidoService.agregarProductoAPedido(
    1L,
    2,
    productoService.buscarProductoPorId(
        1L));

pedidoService.agregarProductoAPedido(
    1L,
    1,
    productoService.buscarProductoPorId(
        2L));

System.out.println(
    "\n===== PEDIDOS =====");

pedidoService.listarPedidos();

System.out.println(
    "\n===== DETALLES PEDIDO =====");

pedidoService.buscarPedidoPorId(
    1L)
    .mostrarDetalles();

} catch (
    EntidadNoEncontradaException
    | StockInvalidoException
    | EmailDuplicadoException e) {

    System.out.println(
        "ERROR: "
        + e.getMessage());

}

}
}

```

Ilustración 12 Prueba integral del sistema

```

run:

===== CATEGORIAS =====
Categoria{id=1, nombre='Bebidas', descripcion='Gaseosas y jugos'}
Categoria{id=2, nombre='Comidas', descripcion='Comidas r pidas'}

===== PRODUCTOS =====
Producto{id=1, nombre='Coca Cola', precio=1500.0, stock=20, categoria=Bebidas}
Producto{id=2, nombre='Sprite', precio=1400.0, stock=10, categoria=Bebidas}

===== USUARIOS =====
Usuario{id=1, nombre='Tomas', email='tomas@gmail.com', rol=ADMIN}
Usuario{id=2, nombre='Juan', email='juan@gmail.com', rol=USUARIO}

===== PEDIDOS =====
Pedido{id=1, fecha=2026-06-21, estado=PENDIENTE, formaPago=TARJETA, total=4400.0, usuario=Tomas}

===== DETALLES PEDIDO =====
DetallePedido{id=1, producto=Coca Cola, cantidad=2, subtotal=3000.0}
DetallePedido{id=2, producto=Sprite, cantidad=1, subtotal=1400.0}
BUILD SUCCESSFUL (total time: 0 seconds)

```

Ilustraci n 13 salida por consola

2.3.6 Cambio de estado del pedido

Figura 14 y 15.

Se verific  el comportamiento avanzado de la entidad Pedido. Se realiz  un cambio de estado, una b squeda de pedidos asociados a un usuario espec fico y una eliminaci n l gica. Estas pruebas permitieron validar las relaciones entre Usuario, Pedido y DetallePedido, as  como el correcto funcionamiento de los servicios implementados.

```

        System.out.println(
            "\n===== CAMBIO DE ESTADO =====");

        pedidoService.cambiarEstadoPedido(
            1L,
            Estado.CONFIRMADO);

        pedidoService.listarPedidos();

        System.out.println(
            "\n===== BUSQUEDA POR USUARIO =====");

        Usuario usuarioBuscado =
            usuarioService.buscarUsuarioPorId(
                1L);

        for (var pedido :
            pedidoService.buscarPedidosPorUsuario(
                usuarioBuscado)) {

            System.out.println(
                pedido);

        }

        System.out.println(
            "\n===== ELIMINACION PEDIDO =====");

        pedidoService.eliminarPedido(
            1L);

        pedidoService.listarPedidos();
    }

```

Ilustraci n 14 Final de validaci n

```

run:

===== CATEGORIAS =====
Categoria{id=1, nombre='Bebidas', descripcion='Gaseosas y jugos'}
Categoria{id=2, nombre='Comidas', descripcion='Comidas rapidas'}

===== PRODUCTOS =====
Producto{id=1, nombre='Coca Cola', precio=1500.0, stock=20, categoria=Bebidas}
Producto{id=2, nombre='Sprite', precio=1400.0, stock=10, categoria=Bebidas}

===== USUARIOS =====
Usuario{id=1, nombre='Tomas', apellido='Lucas', email='tomas@gmail.com', celular='2804123456', rol=ADMIN}
Usuario{id=2, nombre='Juan', apellido='Perez', email='juan@gmail.com', celular='2804555555', rol=USUARIO}

===== PEDIDOS =====
Pedido{id=1, fecha=2026-06-21, estado=PENDIENTE, formaPago=TARJETA, total=4400.0, usuario=Tomas}

===== DETALLES PEDIDO =====
DetallePedido{id=1, producto=Coca Cola, cantidad=2, subtotal=3000.0}
DetallePedido{id=2, producto=Sprite, cantidad=1, subtotal=1400.0}
BUILD SUCCESSFUL (total time: 0 seconds)

```

Ilustración 15 salida por consola

2.3.7 Creación y navegación por el menú

Figura 16 Navegación desde el menú principal al módulo de categorías

Se muestra el acceso desde el menú principal al módulo de categorías mediante la selección de opciones por consola. Esta estructura permite organizar las funcionalidades del sistema de manera modular y facilitar la navegación del usuario.

Figura 17. Creación de una categoría

En esta prueba se verifica el alta de una nueva categoría ingresando nombre y descripción. La información es almacenada en memoria utilizando la clase CategoriaService.

Figura 18. Creación de un producto asociado a una categoría

Se muestra la creación de un producto ingresando sus datos principales, incluyendo nombre, descripción, precio, stock, imagen e identificador de categoría. Esta prueba valida la relación existente entre las entidades Producto y Categoria.

```

===== FOOD STORE =====
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 1

===== MENU CATEGORIAS =====
1. Crear categoría
2. Listar categorías
3. Buscar categoría
4. Actualizar categoría
5. Eliminar categoría
0. Volver
Seleccione una opción: 1
Nombre: gaseosas
Descripción: gasificadas
Categoría creada.

```

Ilustración 16 modulo categorías

```

===== MENU CATEGORIAS =====
1. Crear categoría
2. Listar categorías
3. Buscar categoría
4. Actualizar categoría
5. Eliminar categoría
0. Volver
Seleccione una opción: 0

===== FOOD STORE =====
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 2

```

Ilustración 17 creación categoría

```
===== MENU PRODUCTOS =====
1. Crear producto
2. Listar productos
3. Buscar producto
4. Eliminar producto
0. Volver
Seleccione una opcion: 1
Nombre: Coca Cola
Descripcion: gaseosa
Precio: 3500
Stock: 20
Imagen: coca_cola.jpg
ID Categoria: 1
Producto creado.
```

Ilustración 18 creación de producto en categoría

2.3.8 Navegación por menú

Figura 19 Prueba inicial de navegación del menú

Primero utilicé mensajes simples con `System.out.println()` para verificar que las opciones del menú funcionaran correctamente y que la navegación entre los distintos módulos fuera la esperada. Esta etapa permitió validar la estructura general de la interfaz antes de implementar los métodos específicos de categorías, productos, usuarios y pedidos.

```
case 3:
    System.out.println(
        "Menu Usuarios");
    break;

case 4:
    System.out.println(
        "Menu Pedidos");
    break;

case 0:
    System.out.println(
        "Saliendo...");
```

Ilustración 16 prueba inicial de menú

Figura 19 Prueba final de navegación del menú

Una vez comprobado el correcto funcionamiento de la estructura del menú, los mensajes de prueba fueron reemplazados por llamadas a los métodos específicos de cada módulo. De esta manera, cada opción pasó a ejecutar las funcionalidades reales del sistema, permitiendo acceder a la gestión de categorías, productos, usuarios y pedidos.

```
case 1:
    menuCategorias(scanner);
    break;

case 2:
    menuProductos(scanner);
    break;

case 3:
    menuUsuarios(scanner);
    break;
```

Ilustración 17 prueba final de menú

2.3.10 Creación de usuario

Figura 18. Creación de un usuario

En esta prueba se verificó el registro de un nuevo usuario mediante el ingreso de sus datos personales, incluyendo nombre, apellido, correo electrónico, celular, contraseña y rol. La información fue almacenada correctamente utilizando la clase `UsuarioService`.

```

===== FOOD STORE =====
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción: 3

===== MENU USUARIOS =====
1. Crear usuario
2. Listar usuarios
3. Buscar usuario
4. Eliminar usuario
0. Volver
Seleccione una opción: 1
Nombre: Tomas
Apellido: Lucas
Email: TomasLucas@email.inventado.com
Celular: 15321254
Password: 1234
1. ADMIN
2. USUARIO
Rol: 1
Usuario creado.

```

Ilustración 18 Creación de usuario en consola

2.3.11 Creación de usuario

Figura 19. Búsqueda de usuario

Se comprobó el funcionamiento de la búsqueda de usuarios mediante su identificador único. El sistema recupera y muestra la información correspondiente al usuario solicitado, validando el correcto acceso a los datos almacenados.

```

===== MENU USUARIOS =====
1. Crear usuario
2. Listar usuarios
3. Buscar usuario
4. Eliminar usuario
0. Volver
Seleccione una opción: 3
ID Usuario: 1
Usuario{id=1, nombre='Tomas', apellido='Lucas', email='TomasLucas@email.inventado.com', celular='15321254', rol=ADMIN}

```

Ilustración 19 Búsqueda de usuario en consola

2.3.12 Eliminación de usuario

Figura 20. Eliminación lógica de un usuario

En esta prueba se validó la eliminación lógica de un usuario. En lugar de eliminar físicamente el registro, el sistema modifica el atributo eliminado, evitando que el usuario vuelva a aparecer en los listados activos y preservando la integridad de la información.

```

===== MENU USUARIOS =====
1. Crear usuario
2. Listar usuarios
3. Buscar usuario
4. Eliminar usuario
0. Volver
Seleccione una opción: 4
ID Usuario: 1
Usuario eliminado.

```

Ilustración 20 Eliminación de usuario por consola

Figura 21. Creación de un pedido y carga de productos.

Luego de crear las categorías, productos y usuarios del sistema, se procedió a generar un nuevo pedido asociado a un usuario. Posteriormente se agregaron productos indicando la cantidad deseada. Esta prueba permitió verificar la relación entre las entidades Usuario, Pedido, Producto y DetallePedido, así como el cálculo automático del total mediante la interfaz Calculable.

```
===== MENU PEDIDOS =====
1. Crear pedido
2. Agregar producto
3. Listar pedidos
4. Cambiar estado
5. Eliminar pedido
0. Volver
Seleccione una opcion: 1
ID Usuario: 1
1. EFECTIVO
2. TARJETA
3. TRANSFERENCIA
Forma de pago: 2
Pedido creado.

===== MENU PEDIDOS =====
1. Crear pedido
2. Agregar producto
3. Listar pedidos
4. Cambiar estado
5. Eliminar pedido
0. Volver
Seleccione una opcion: 2
ID Pedido: 1
ID Producto: 1
Cantidad: 2
Producto agregado.
```

Ilustración 21 creación de pedido en consola

2.3.14 Listado de pedidos

Figura 22. Listado de pedidos

Se verificó la correcta visualización de los pedidos registrados en el sistema. El listado muestra información relevante como el identificador, fecha de creación, estado actual, forma de pago, total calculado y usuario asociado.

```
===== MENU PEDIDOS =====
1. Crear pedido
2. Agregar producto
3. Listar pedidos
4. Cambiar estado
5. Eliminar pedido
0. Volver
Seleccione una opcion: 3
Pedido{id=1, fecha=2026-06-21, estado=PENDIENTE, formaPago=TARJETA, total=7000.0, usuario=Tomas}
```

Ilustración 22 Listado de pedidos por consola

Figura 23 y 24. Cambio de estado de un pedido

En esta prueba se validó la actualización del estado de un pedido utilizando el enumerado Estado. El sistema permite modificar el estado durante su ciclo de vida, pasando por etapas como Pendiente, Confirmado, En preparación, Enviado, Entregado o Cancelado.

```
===== MENU PEDIDOS =====
1. Crear pedido
2. Agregar producto
3. Listar pedidos
4. Cambiar estado
5. Eliminar pedido
0. Volver
Seleccione una opcion: 4
ID Pedido: 1
1. PENDIENTE
2. CONFIRMADO
3. EN_PREPARACION
4. ENVIADO
5. ENTREGADO
6. CANCELADO
Seleccione el nuevo estado: 2
Estado actualizado.
```

Ilustración 23 Cambio de estado de pedido por consola

```
===== MENU PEDIDOS =====
1. Crear pedido
2. Agregar producto
3. Listar pedidos
4. Cambiar estado
5. Eliminar pedido
0. Volver
Seleccione una opcion: 3
Pedido{id=1, fecha=2026-06-21, estado=CONFIRMADO, formaPago=TARJETA, total=7000.0, usuario=Tomas}
```

Ilustración 24 Muestra de estado de pedido por consola

2.3.16 Eliminación de pedido

Figura 25. Eliminación lógica de un pedido

Se comprobó el funcionamiento de la eliminación lógica de pedidos. Al eliminar un pedido, este deja de mostrarse en los listados activos, aunque la información permanece almacenada internamente mediante el atributo eliminado heredado de la clase Base.

```
===== MENU PEDIDOS =====
1. Crear pedido
2. Agregar producto
3. Listar pedidos
4. Cambiar estado
5. Eliminar pedido
0. Volver
Seleccione una opcion: 5
ID Pedido: 1
Pedido eliminado.

===== MENU PEDIDOS =====
1. Crear pedido
2. Agregar producto
3. Listar pedidos
4. Cambiar estado
5. Eliminar pedido
0. Volver
Seleccione una opcion: 3
```

Ilustración 25 Eliminación de pedido por consola

3. Dificultades y soluciones.

Durante el desarrollo del proyecto, una de las dificultades que más me costó fue organizar correctamente las clases y entender cómo relacionarlas entre sí. Al principio me resultó confuso decidir qué responsabilidades debía tener cada entidad y cuáles debían estar dentro de los servicios. Con el avance del trabajo fui separando mejor la lógica de negocio de las entidades, logrando una estructura más ordenada.

Otra dificultad importante fue la implementación de la herencia y la interfaz Calculable. Me llevó tiempo comprender por qué todas las entidades debían heredar de la clase Base y cómo aprovechar esa estructura para reutilizar atributos comunes. También tuve que realizar varias pruebas hasta lograr que el cálculo automático de los pedidos funcionara correctamente al agregar productos.

Por último, la gestión de pedidos fue una de las partes que más trabajo me demandó, ya que involucraba relacionar usuarios, productos y detalles de pedido. A través de distintas pruebas y correcciones logré implementar correctamente la creación de pedidos, el agregado de productos, el cambio de estado y la eliminación lógica.

En general, los problemas fueron resolviéndose mediante pruebas constantes, corrección de errores y la consulta de apuntes y documentación de Java cuando fue necesario.

4. Bibliografía / Webgrafía.

Durante el desarrollo del proyecto se consultaron distintas fuentes de información y material de apoyo para reforzar conceptos de Programación Orientada a Objetos y Java.

Material de la materia

- Apuntes, presentaciones y material teórico proporcionado en el Aula Virtual de Programación 2.
- Consignas y documentación entregadas durante la cursada.

Cursos y material complementario

- CodeJa.dev – Curso de Java y Programación.
<https://www.codeja.dev/courses/curso-de-java-introduccion-a-la-programacion-java/lesson/que-es-java>
- Canal de YouTube de Mariano Cimino – Contenido sobre Java, Programación Orientada a Objetos y desarrollo de aplicaciones.
<https://www.youtube.com/@CharlyCimino>

Herramientas utilizadas

- Apache NetBeans IDE.
<https://netbeans.apache.org/>
- GitHub.
<https://github.com/>

5. Repositorio y video explicativo.

En esta sección se incluyen los enlaces correspondientes al repositorio del proyecto y al video explicativo solicitado para la entrega.

Repositorio GitHub

Link al repositorio:

<https://github.com/TomasEmmanuelLucas/FoodStore.git>

Video Explicativo

Link al video:

<https://youtu.be/5XLvNWh8S9w>